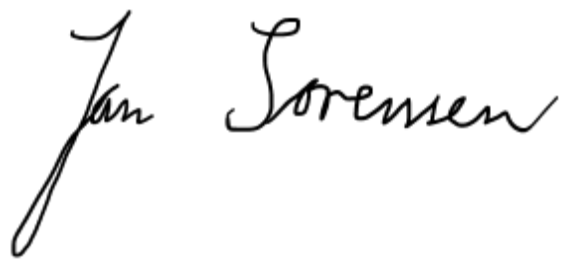
JPMS Layers



Agenda

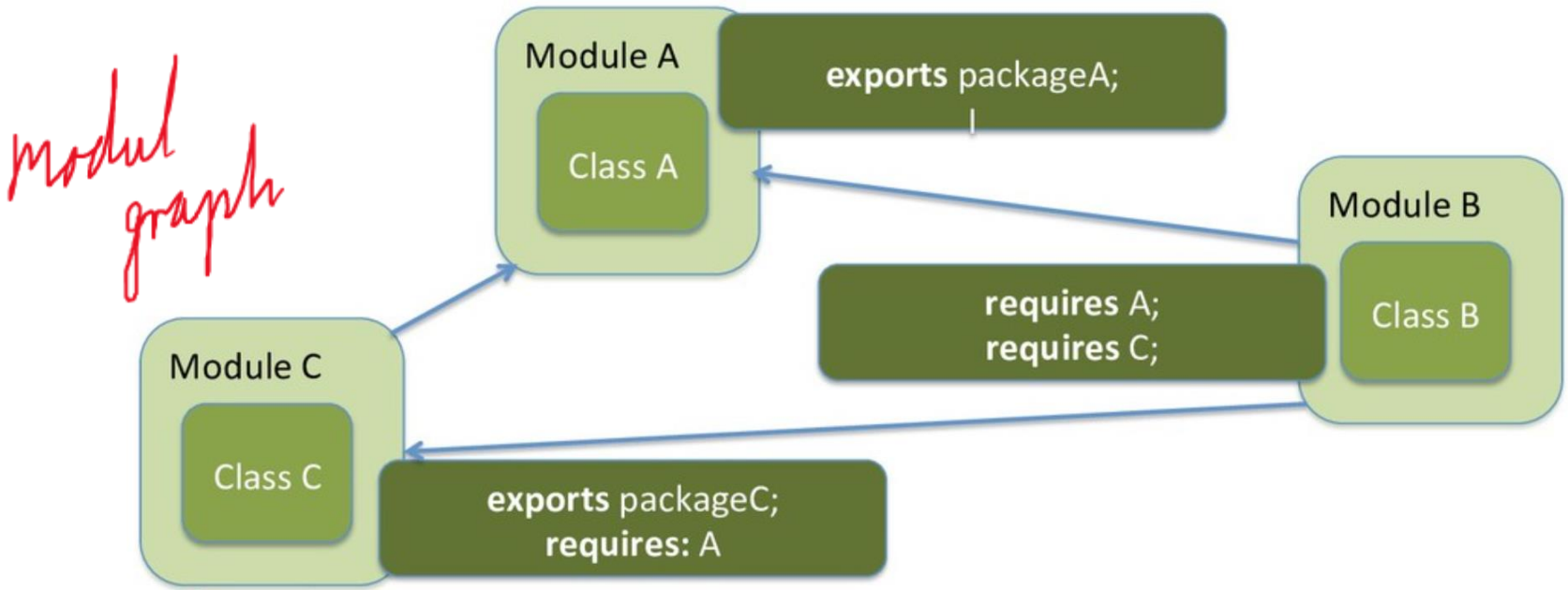
- Short about JPMS
- Jar Hell problem
- Versioning as a solution to Jar hell
- JPMS Layers
- Jar hell resolution using Layers

Java Platform Module System

Jigsaw module

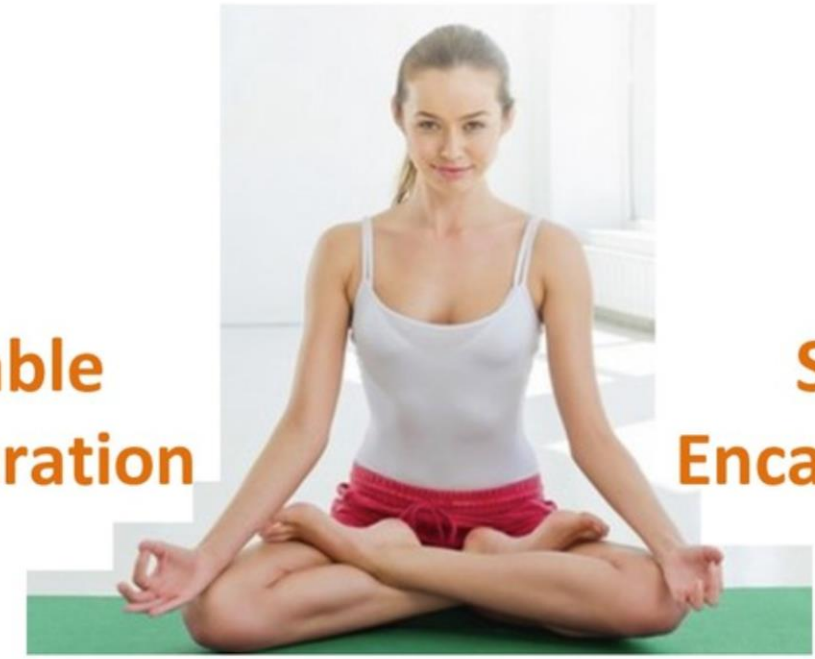
- imports:
  - modules: **requires**
  - services: **uses**
- exports:
  - packages: **exports**
  - servicies: **provides** SomeService **with** ServiceImpl

Java Platform Module System



Jigsaw Mantra

Reliable Configuration

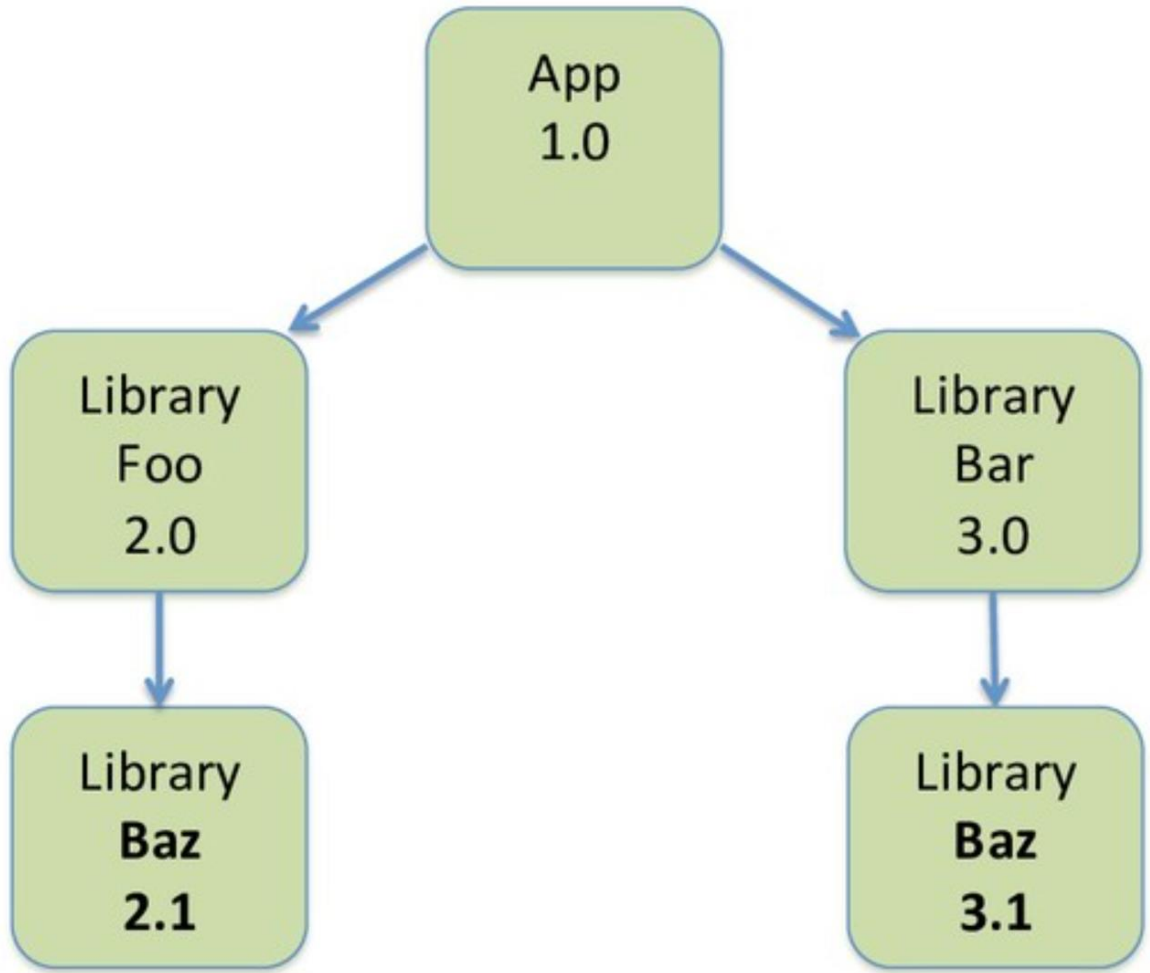


Strong Encapsulation

Strong Encapsulation



Jar/Classpath Hell



Versioning

- Jigsaw module might have a version in the first drafts of Jigsaw to solve Jar Hell problem
  - import/export was qualified by version
  - If two modules require a library module of two different versions, both versions will be loaded

Versioning

However versioning was removed later …

Why?

Versioning

Question: How to implement versioning?

Task: For a given module **A** importing library Lib (**v1**), and module **B** importing Lib (**v2**), it is **required** that both versions of Lib work without conflicts.

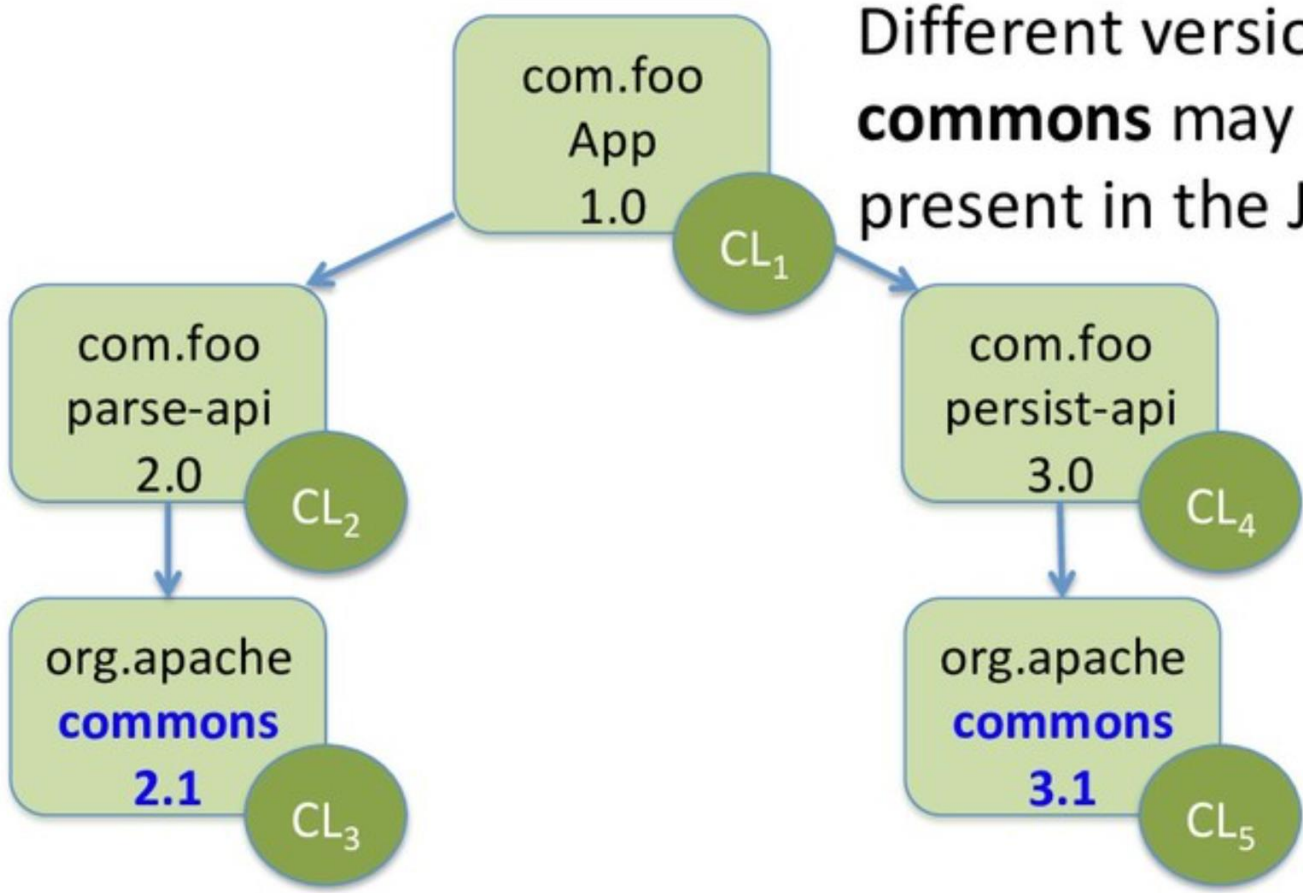
Solution: load versions of the library by different *classloaders*.

Versioning

- ClassLoader forms a ***unique class namespace***

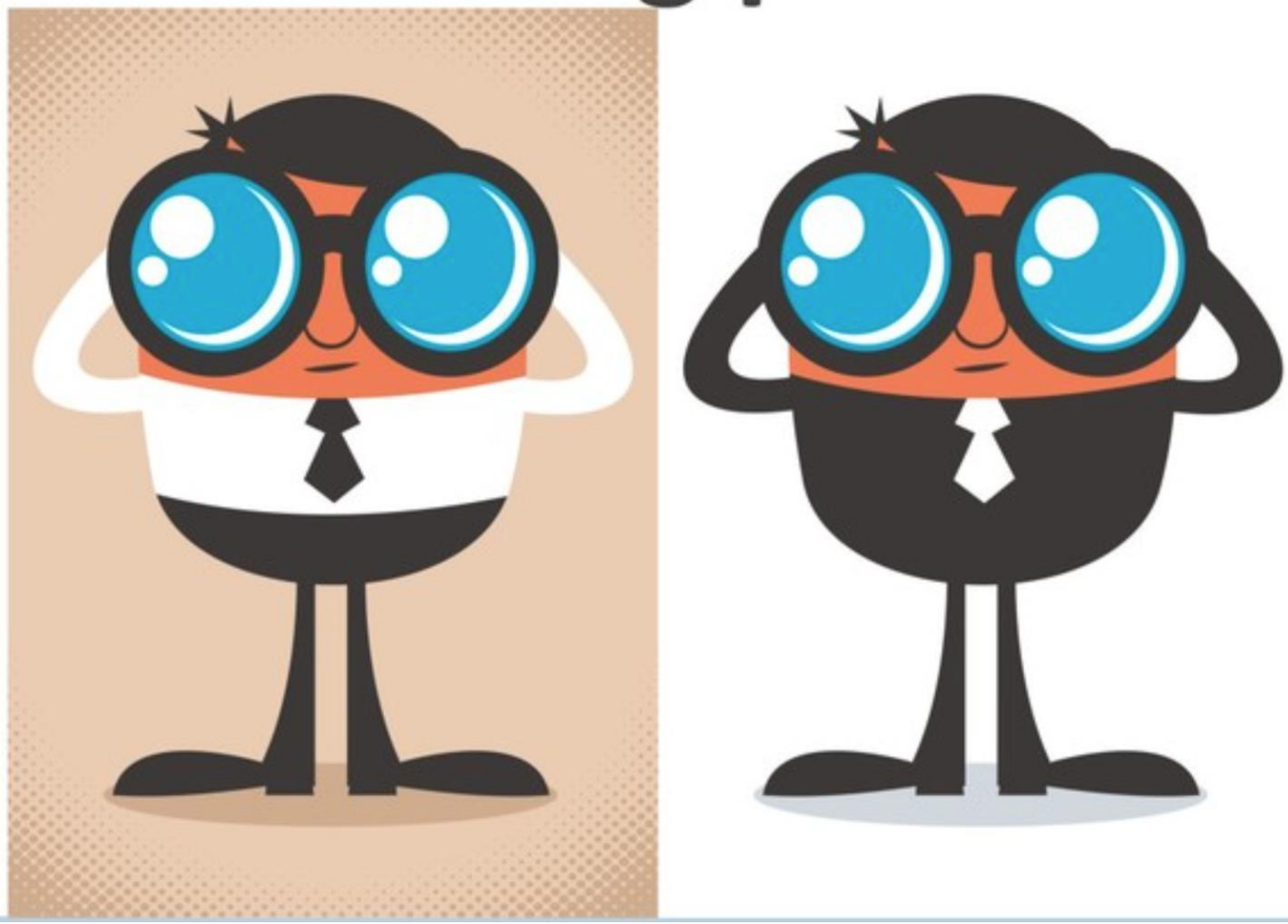
- If each module is loaded by its own classloader then there will be no conflicts with classes of other modules

Versioning



Different versions of **apache commons** may simultaneously present in the JVM

Versioning problems



Versioning

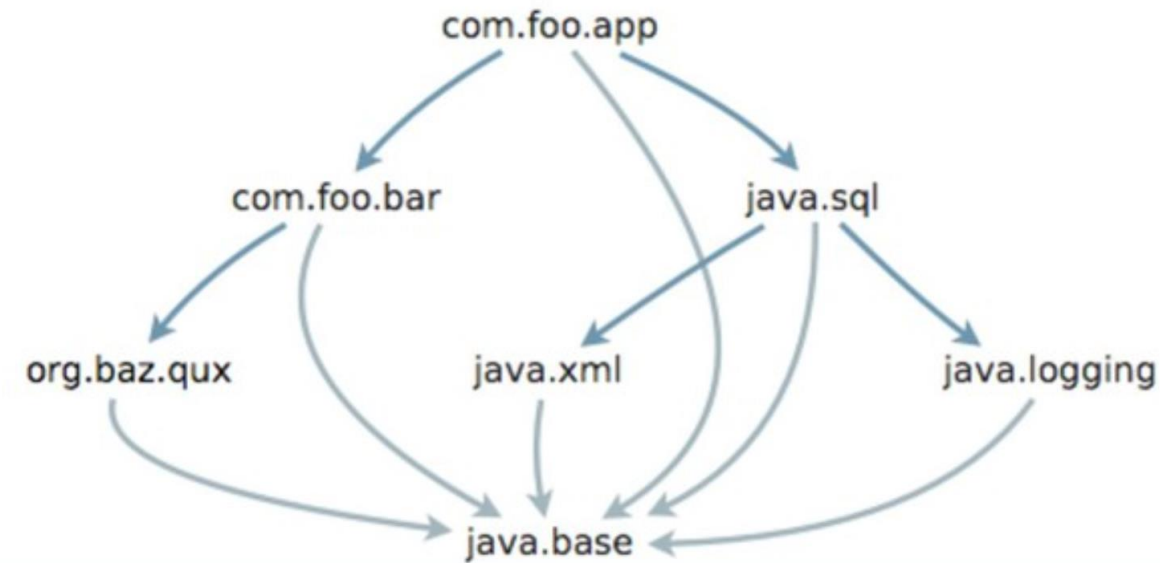
Versioning immediately means:

1 module ←→ 1 classloader

It was exactly so in the first Jigsaw drafts!

Jigsaw and classloaders

Jigsaw is not just modules, but also the Java SE platform split into modules.



Jigsaw and classloaders

Backward compatibility problems:

JVM INTERNAL

According to the specification

CLASSLOADER

getClassloader() == null

for core platform classes.

It precludes splitting of the platform into modules, with each module loaded by its loader

Jigsaw and classloaders

Backward compatibility problems:

Many classloaders for an application may also cause problems:

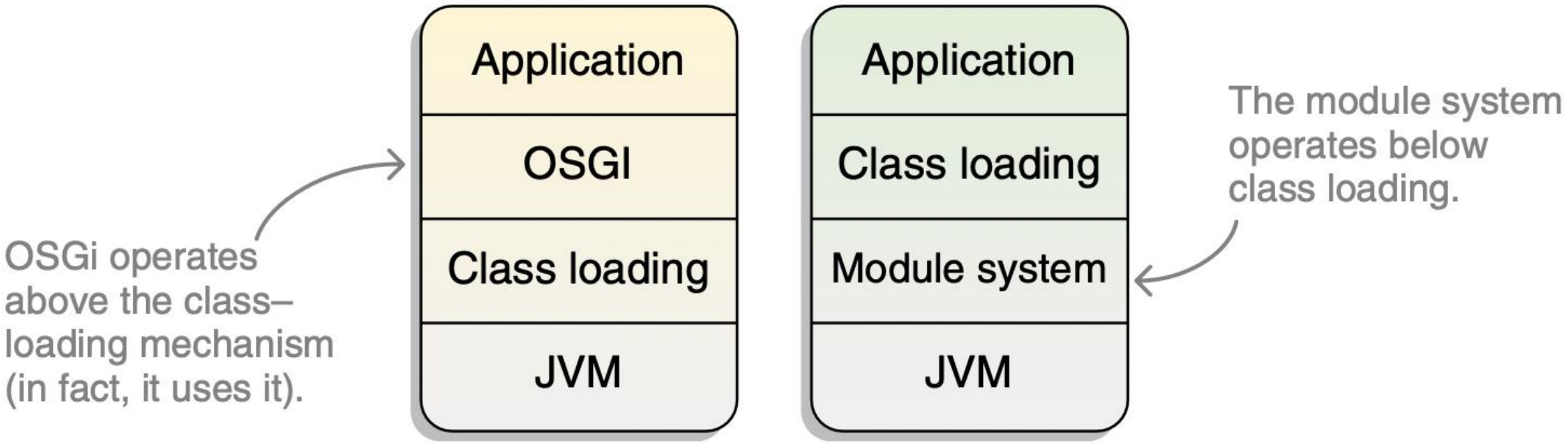
- classloaders delegation

Jigsaw and classloaders

Backward compatibility problems:

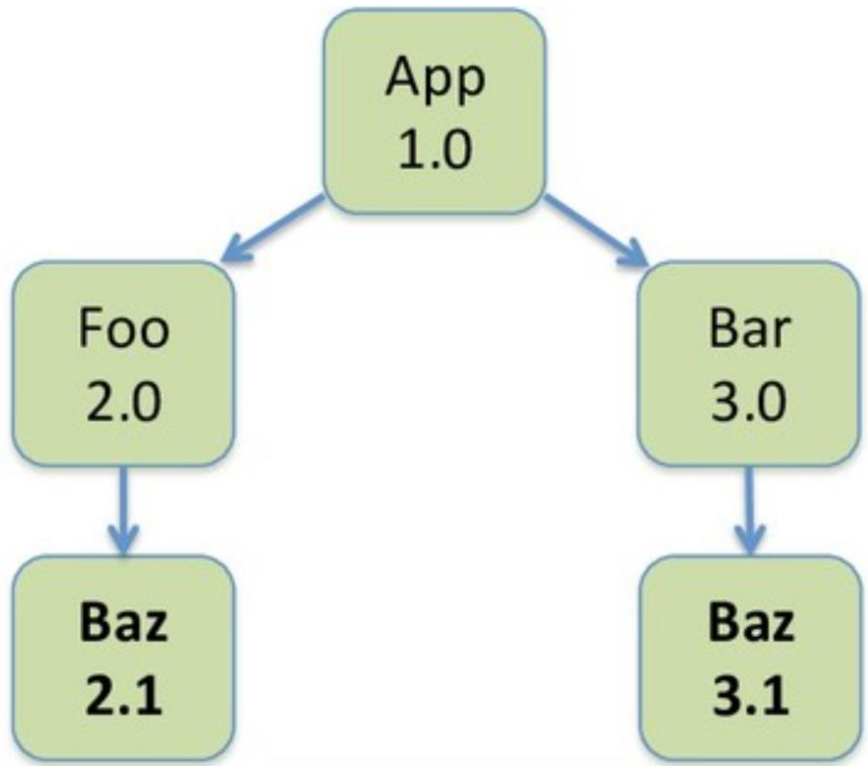
Even the fact that Application (system) Classloader does not extend URLClassloader anymore causes problems

Other module systems VS JPMS



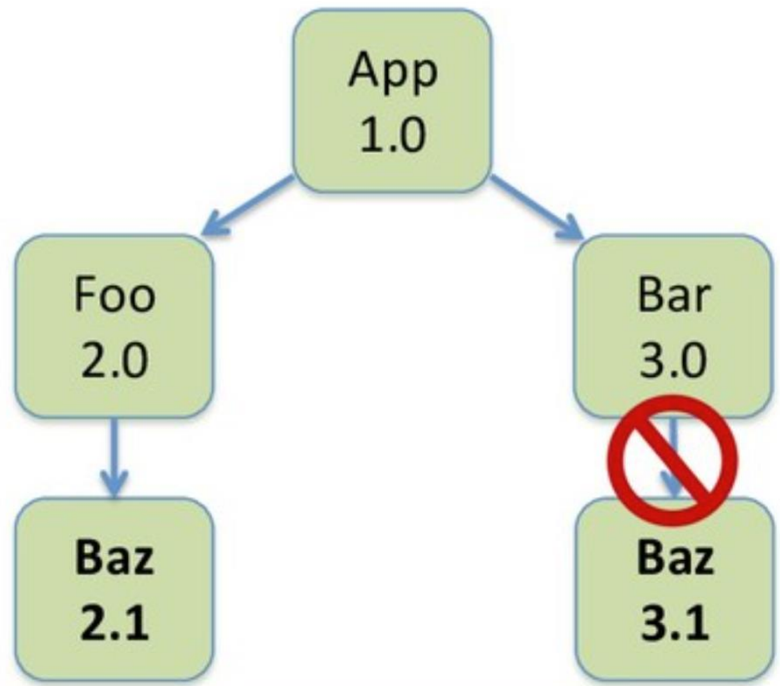
Jar Hell

But how to solve the Jar Hell now?



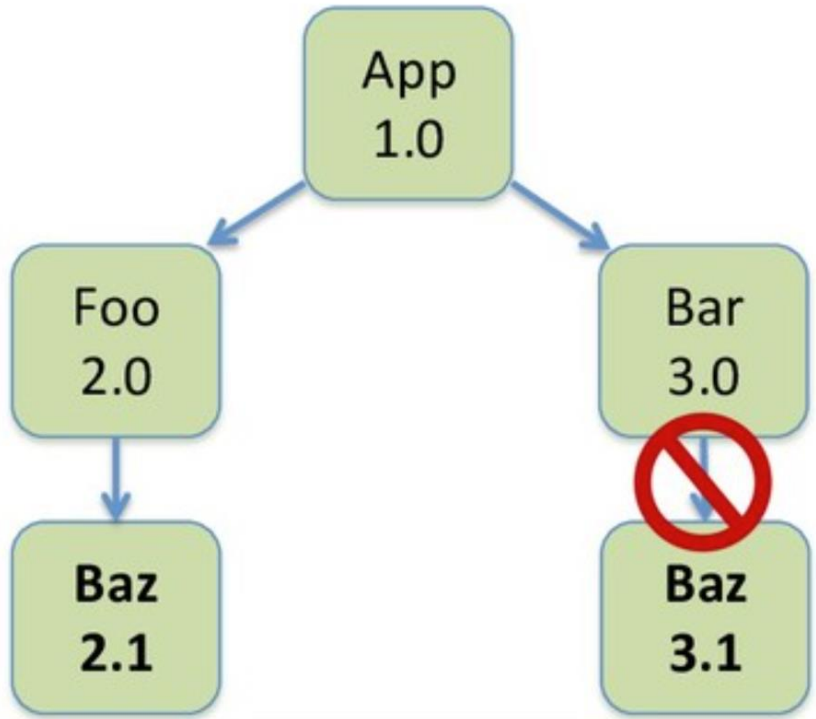
Jar Hell

Jigsaw simply **prohibits** this situation!



Jar Hell

Stop! Jigsaw does not have versions!



Split Package Problem

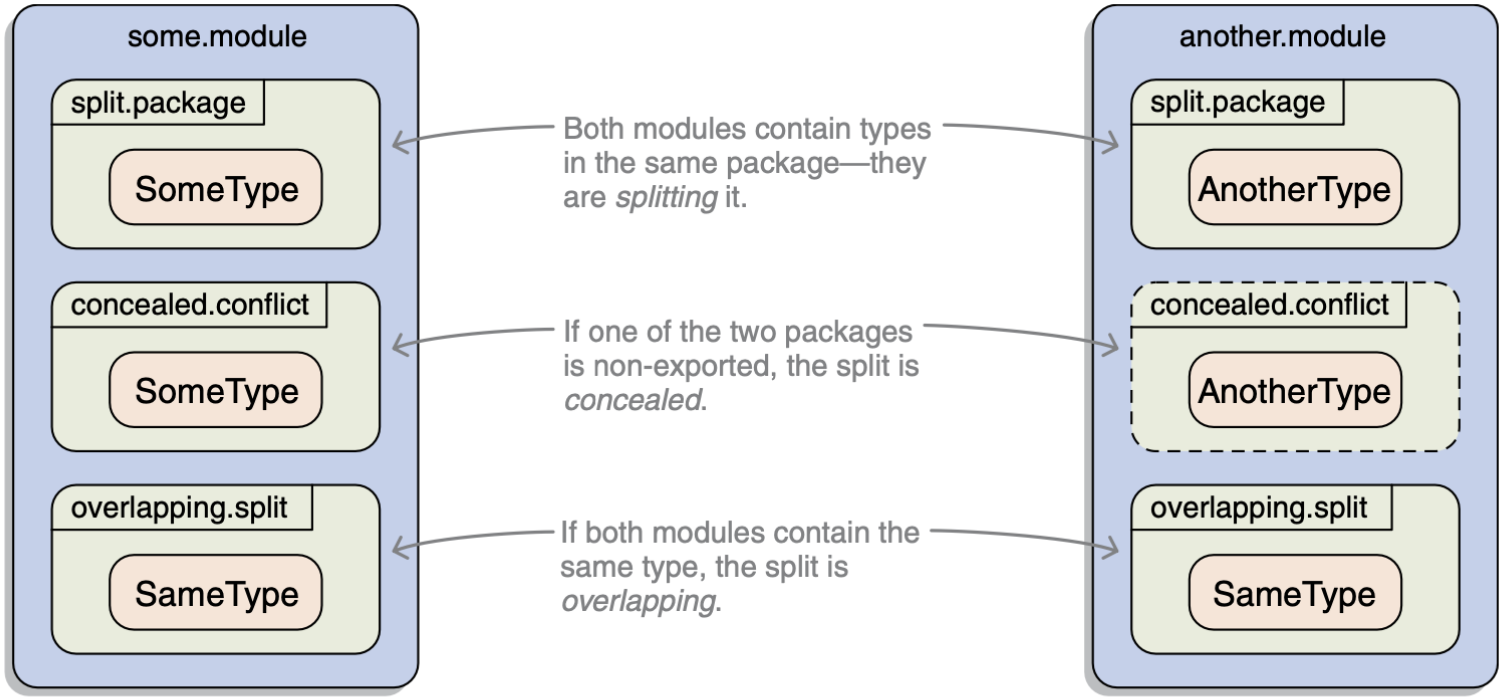


Figure 7.2 When two modules contain types in the same package, they split the package.

Jar Hell

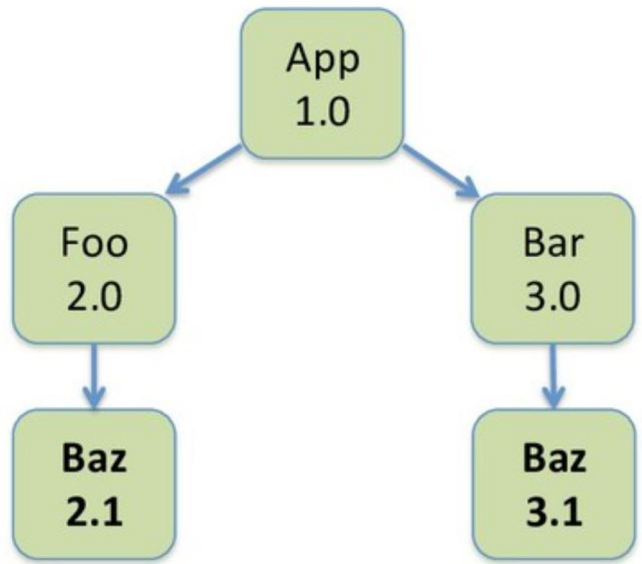
Stop! Jigsaw does not have versions!

The most probably two different versions of the same library have same packages.

Jigsaw prohibits for two modules to have same packages (split packages)

Jar Hell

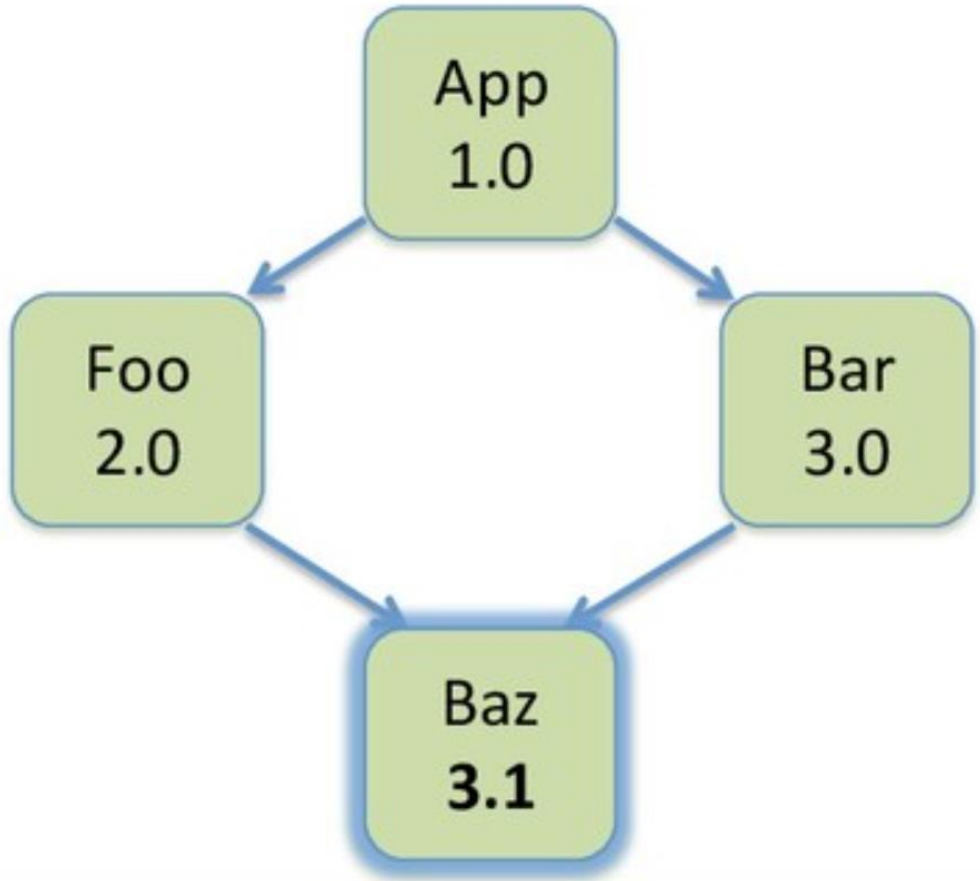
But how to solve the Jar Hell problem **after all**?



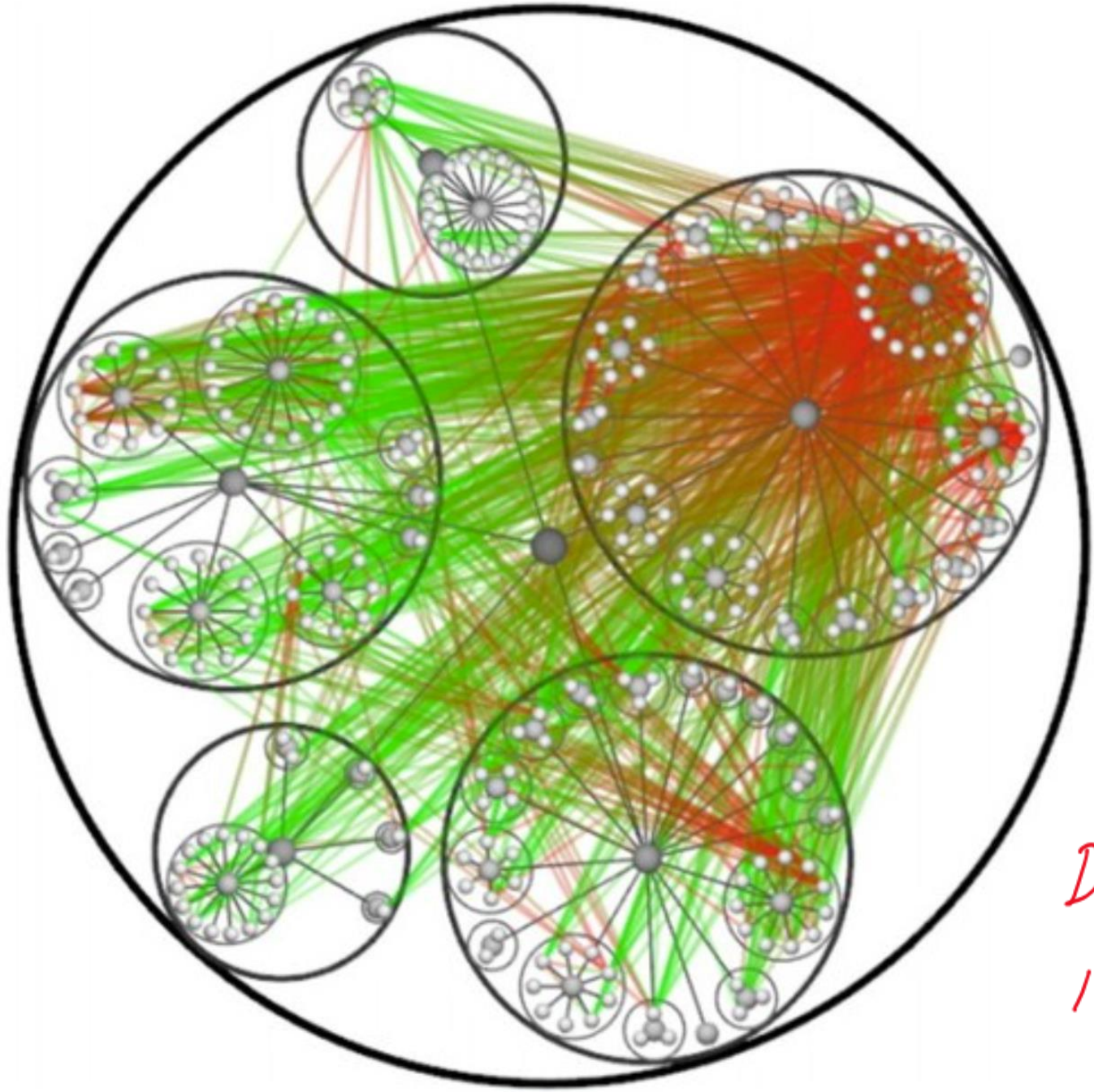
EXCELSIOR

Jar Hell

A possible solution:



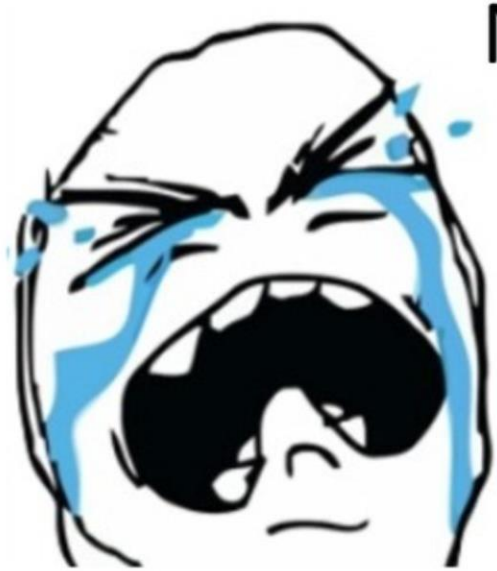
A REAL DEPENDENCY GRAPH



NP COMPLETE

DO NOT SOLVE IT MANUALLY

I always dreamed of solving NP-complete problems!



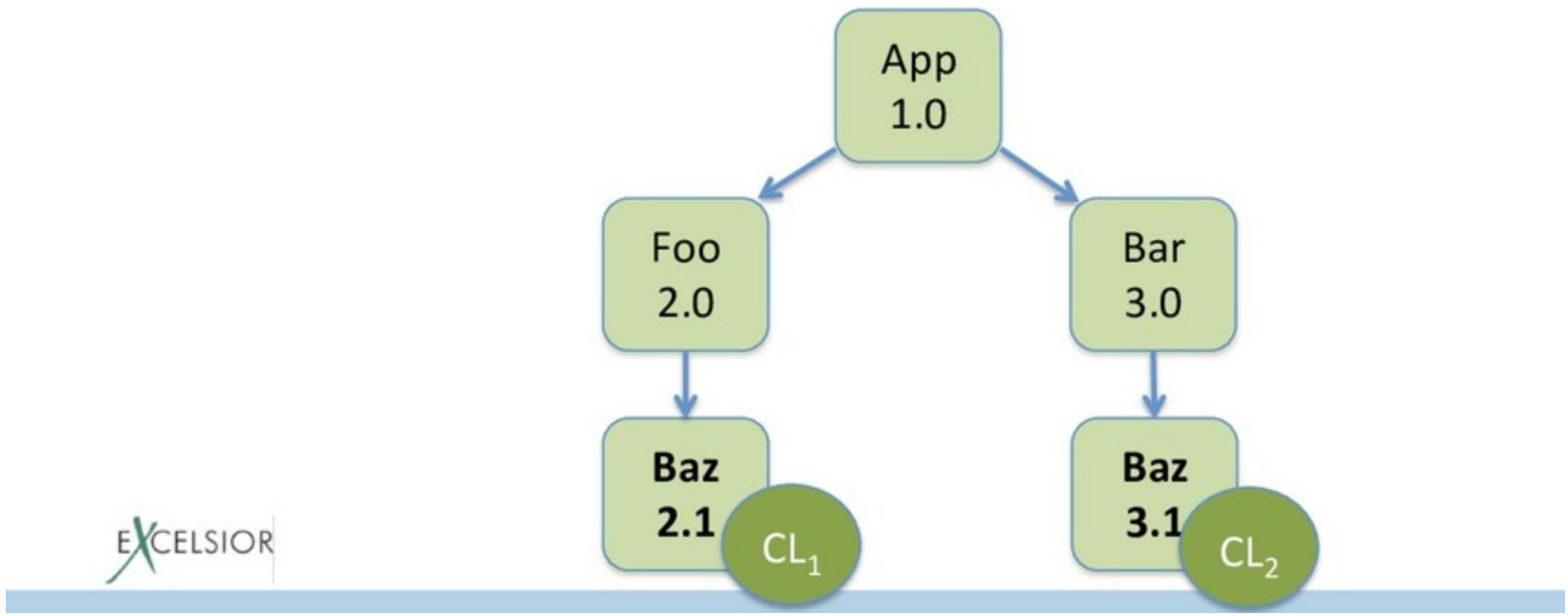
Jar Hell

Another solution: relocate classes of two versions of a library to different Java packages (Maven Shade plugin)

It may not work because of reflection.

Jar Hell

Third solution: employ classloaders



EXCELSIOR

Jar Hell

Third solution: employ classloaders

Be ready to fight with classloaders constraints violation problem



EXCELSIOR


Jigsaw Layers *

Jigsaw Layers

Fourth solution: employ Jigsaw ***Layers***

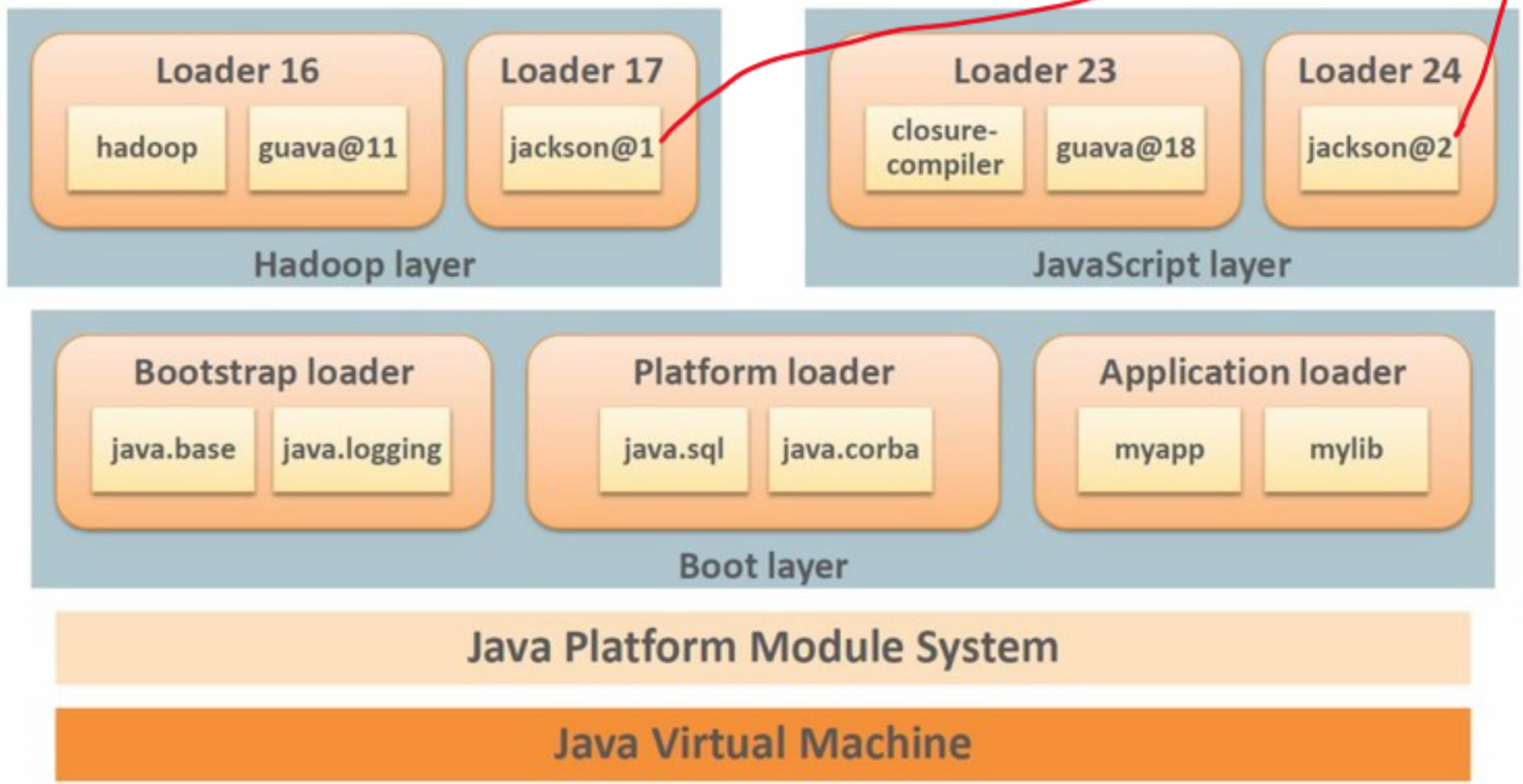


Jigsaw Layers

- Local module system
- Split packages are prohibited inside one layer
- Two modules containing the same package have to belong to different layers

SPLIT PACKAGE

Jigsaw Layers *



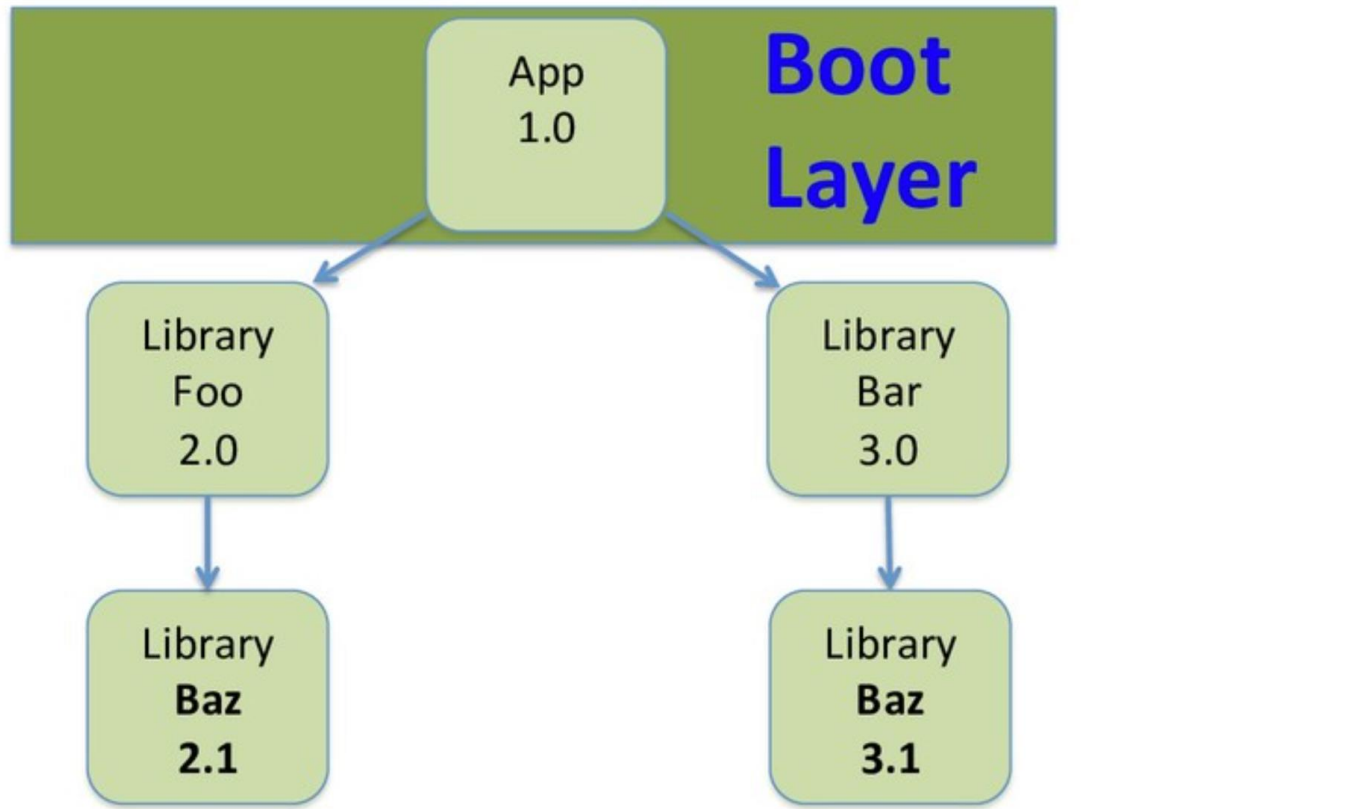
LSIOR

* The picture is from Alex Buckley's presentation: Project Jigsaw Under the hood

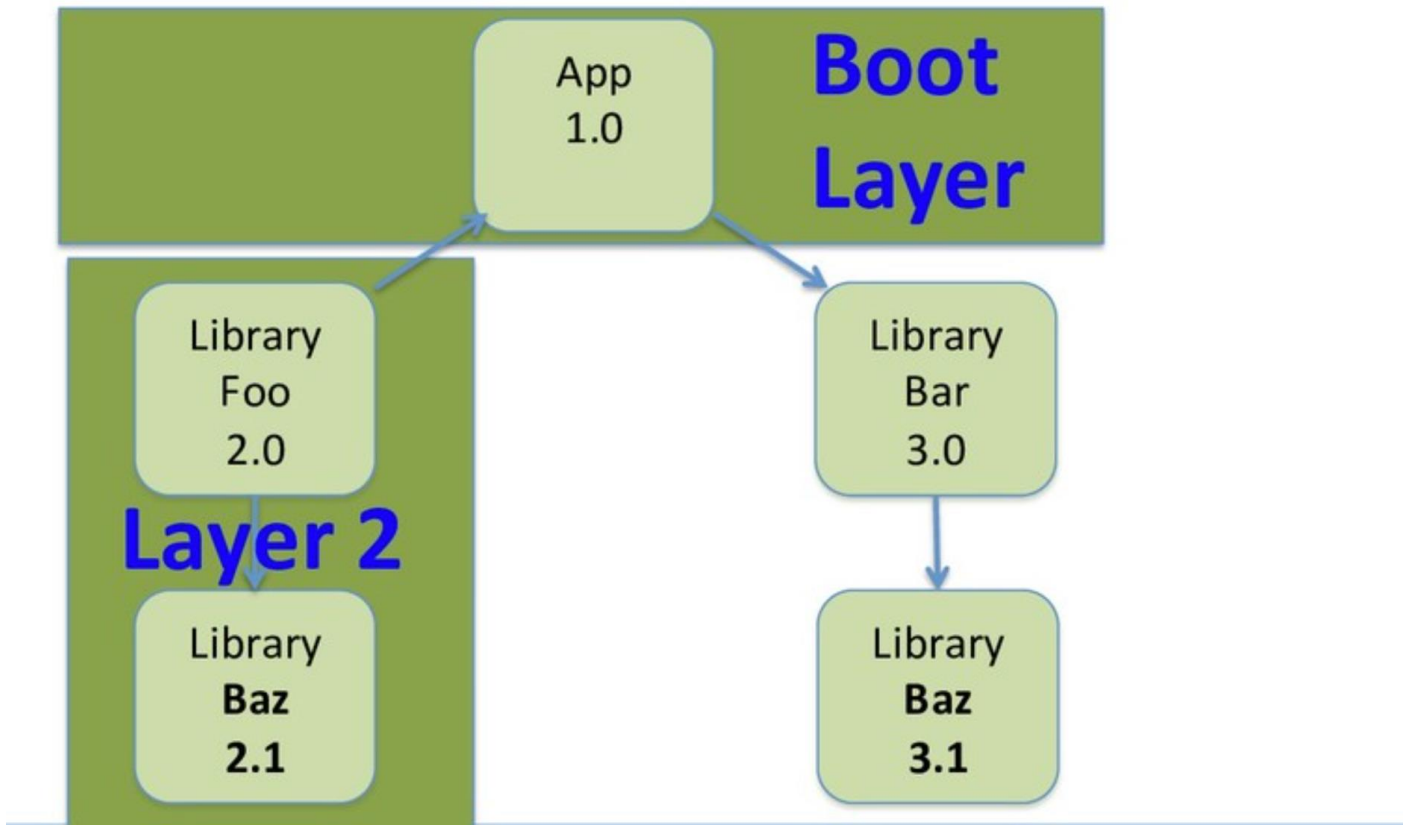
Jigsaw Layers

- Jigsaw layers are implemented via classloaders!
- Each layer has at least one unique classloader (may have several classloaders)
- However, classloader constraints are **impossible** to violate by construction!

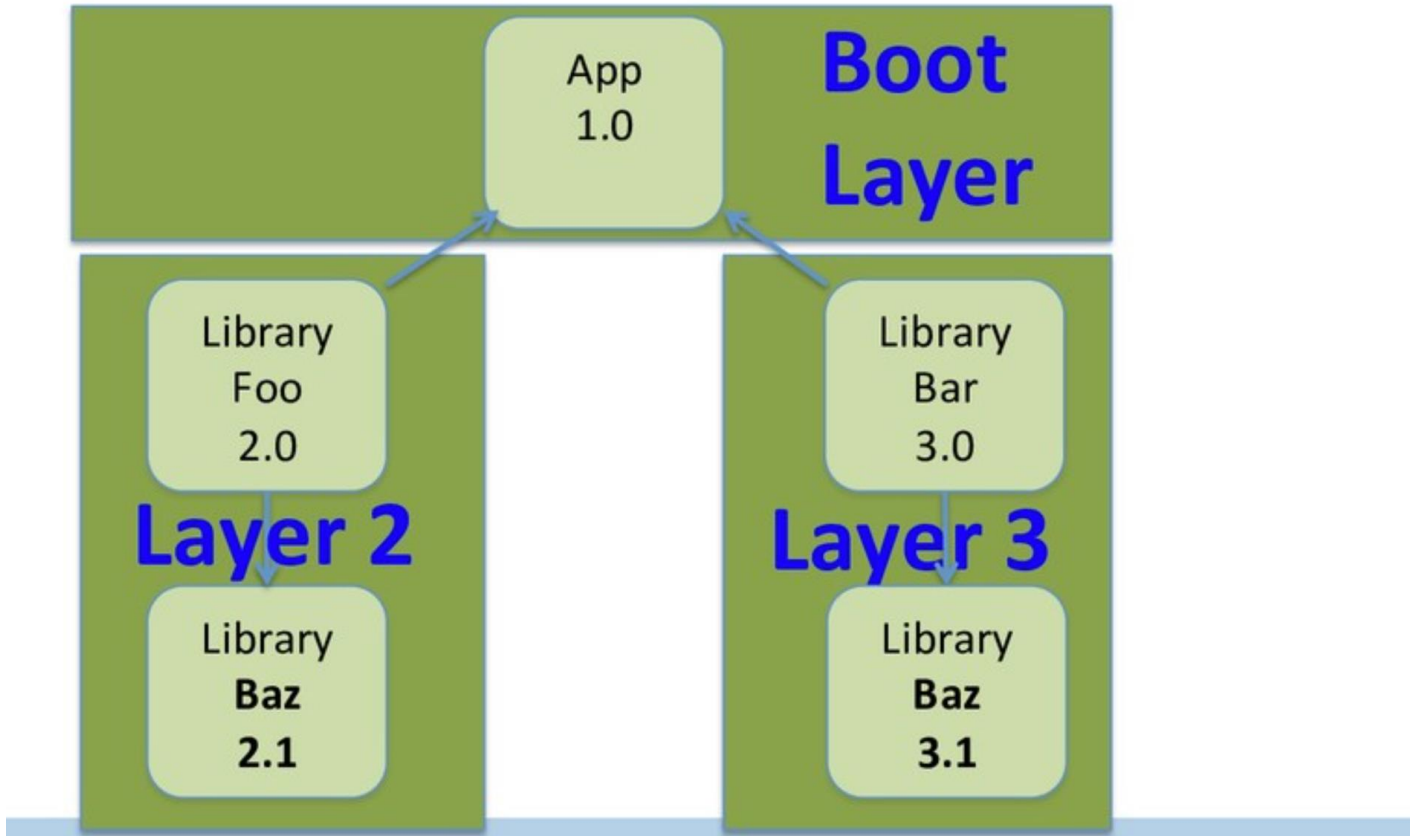
Jigsaw Layers Jar Hell Resolution



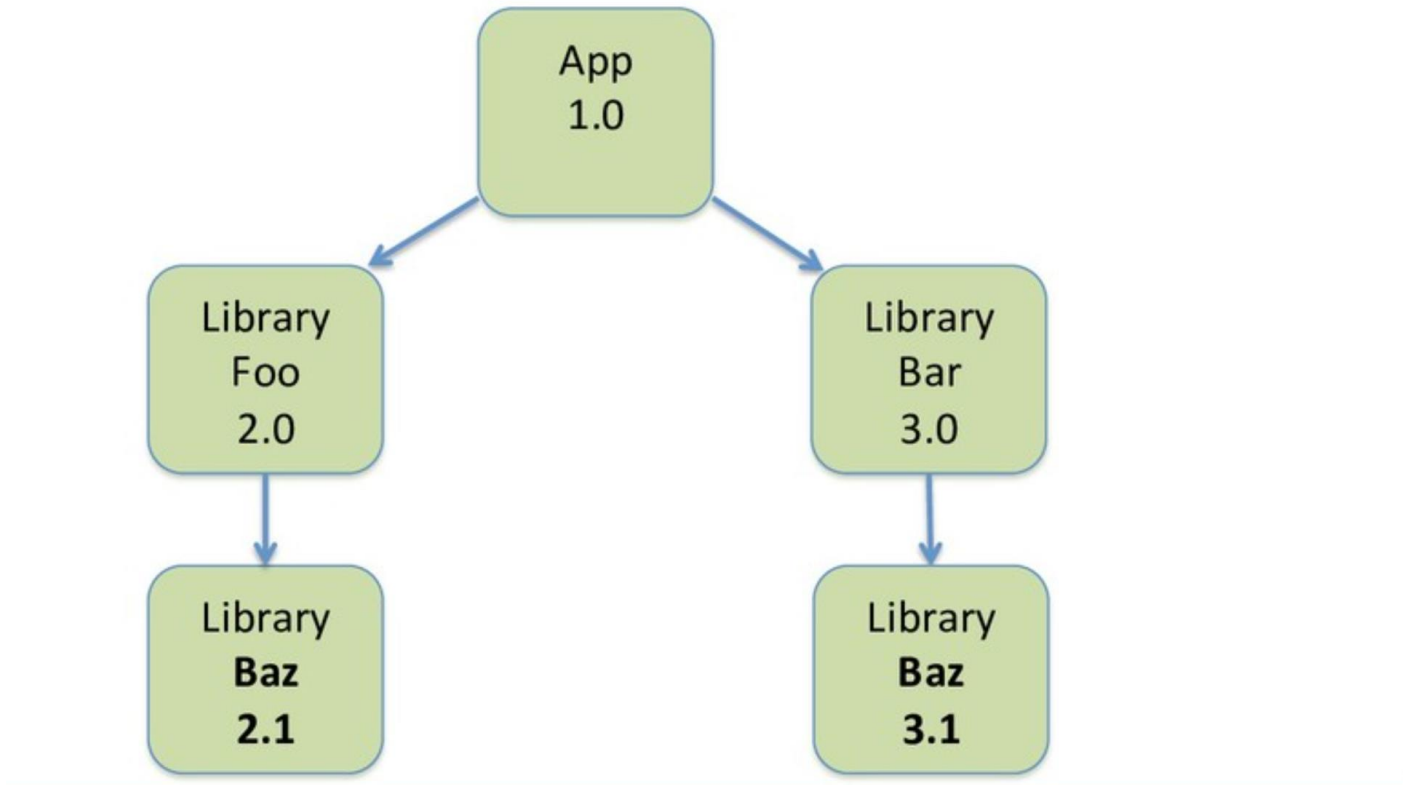
Jigsaw Layers Jar Hell Resolution



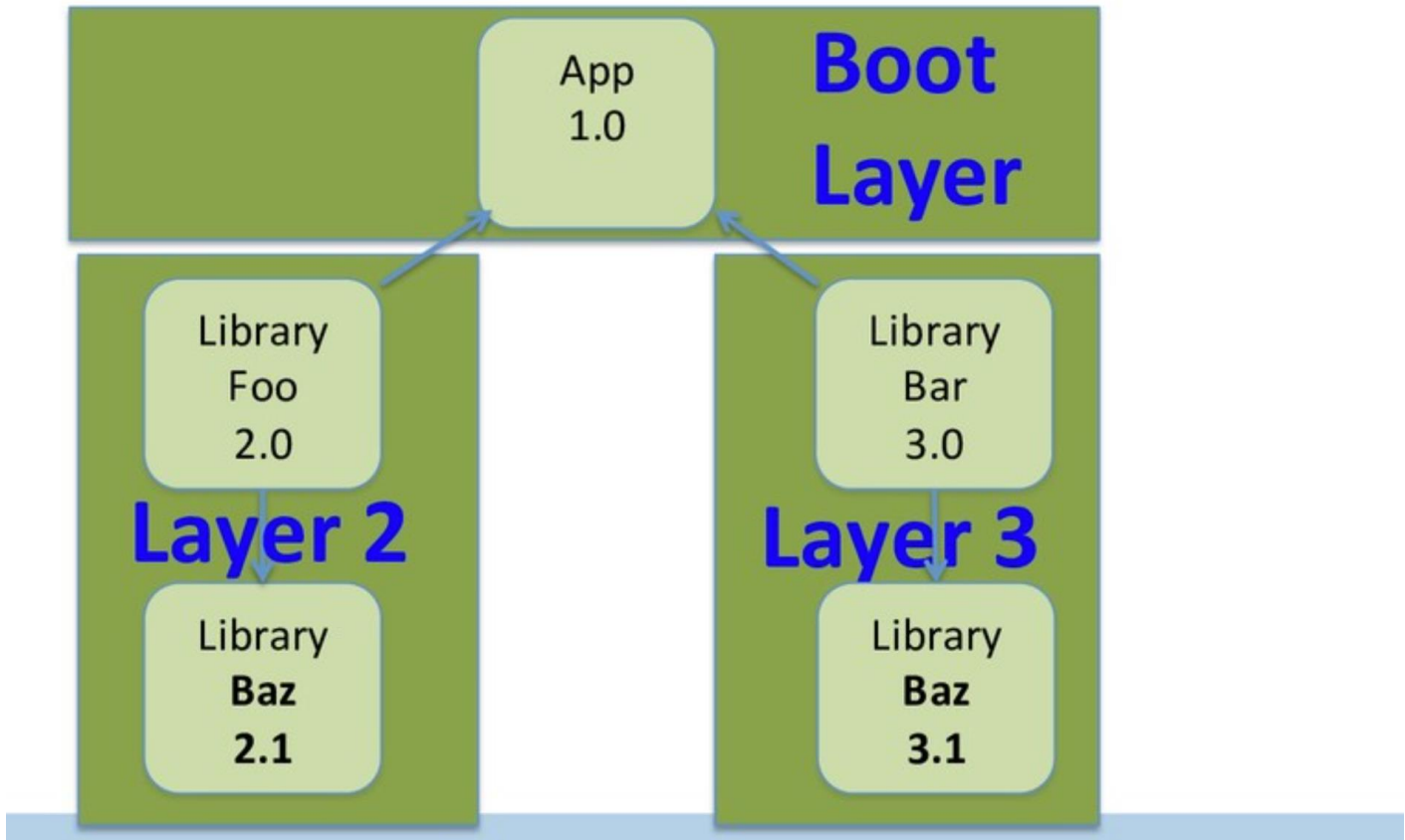
Jigsaw Layers Jar Hell Resolution



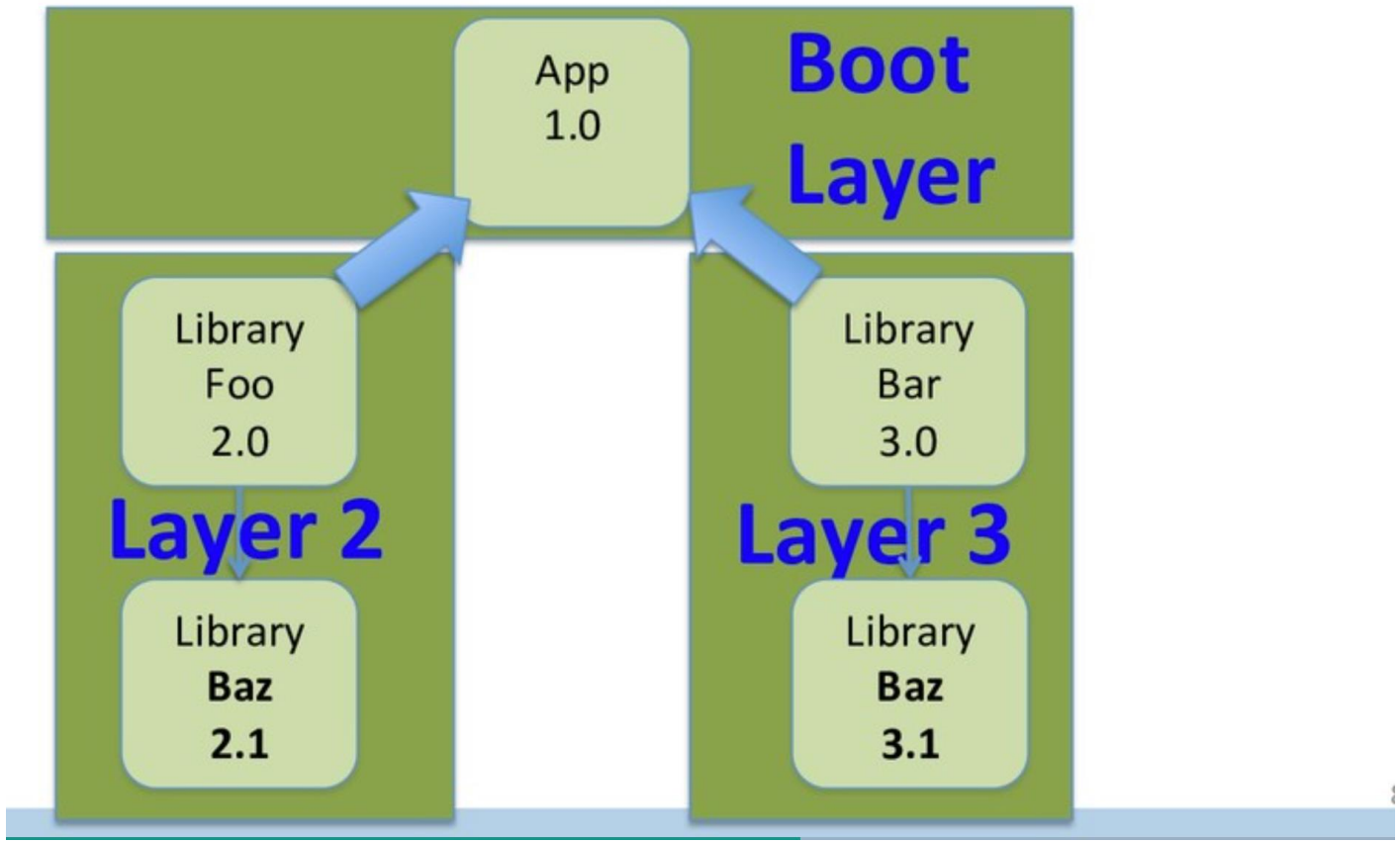
Jigsaw Layers Jar Hell Resolution



Jigsaw Layers Jar Hell Resolution



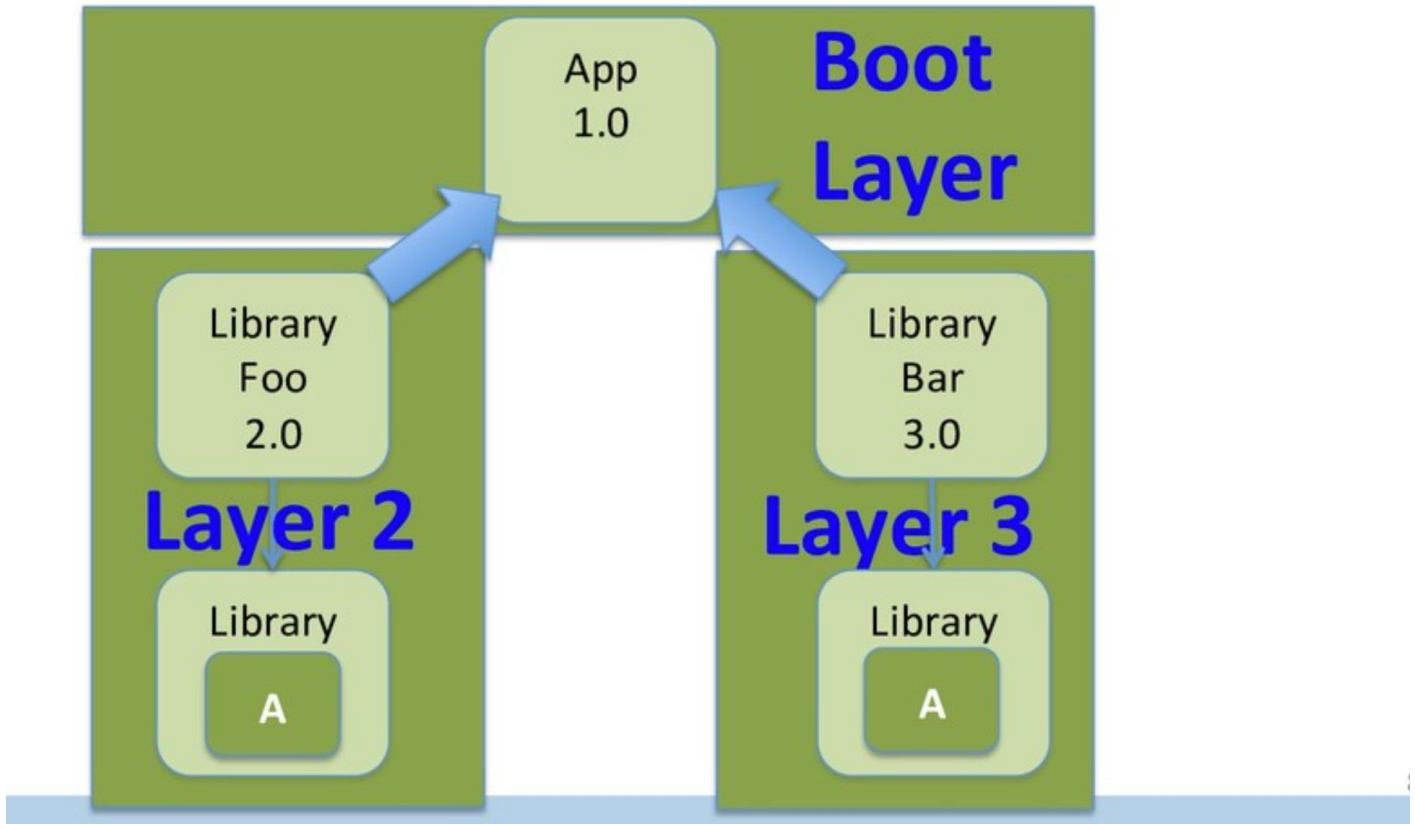
Jigsaw Layers Jar Hell Resolution



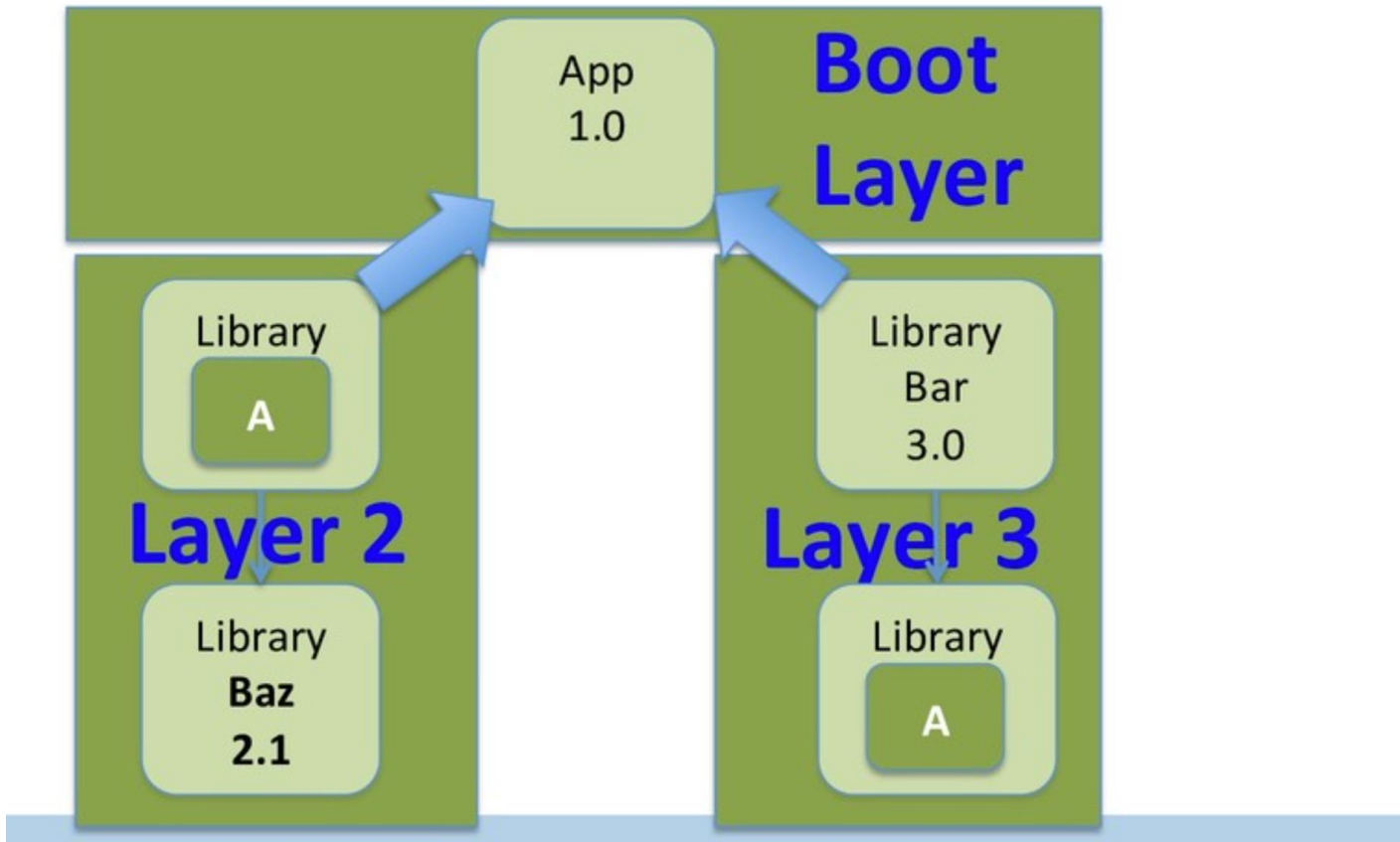
Jigsaw Layers

- Every layer has a parent layer
- Modules of a child layer can import modules of parent layers
- However, parent layer modules cannot import child layers modules!
- That’s why it’s impossible to violate classloaders constraints!

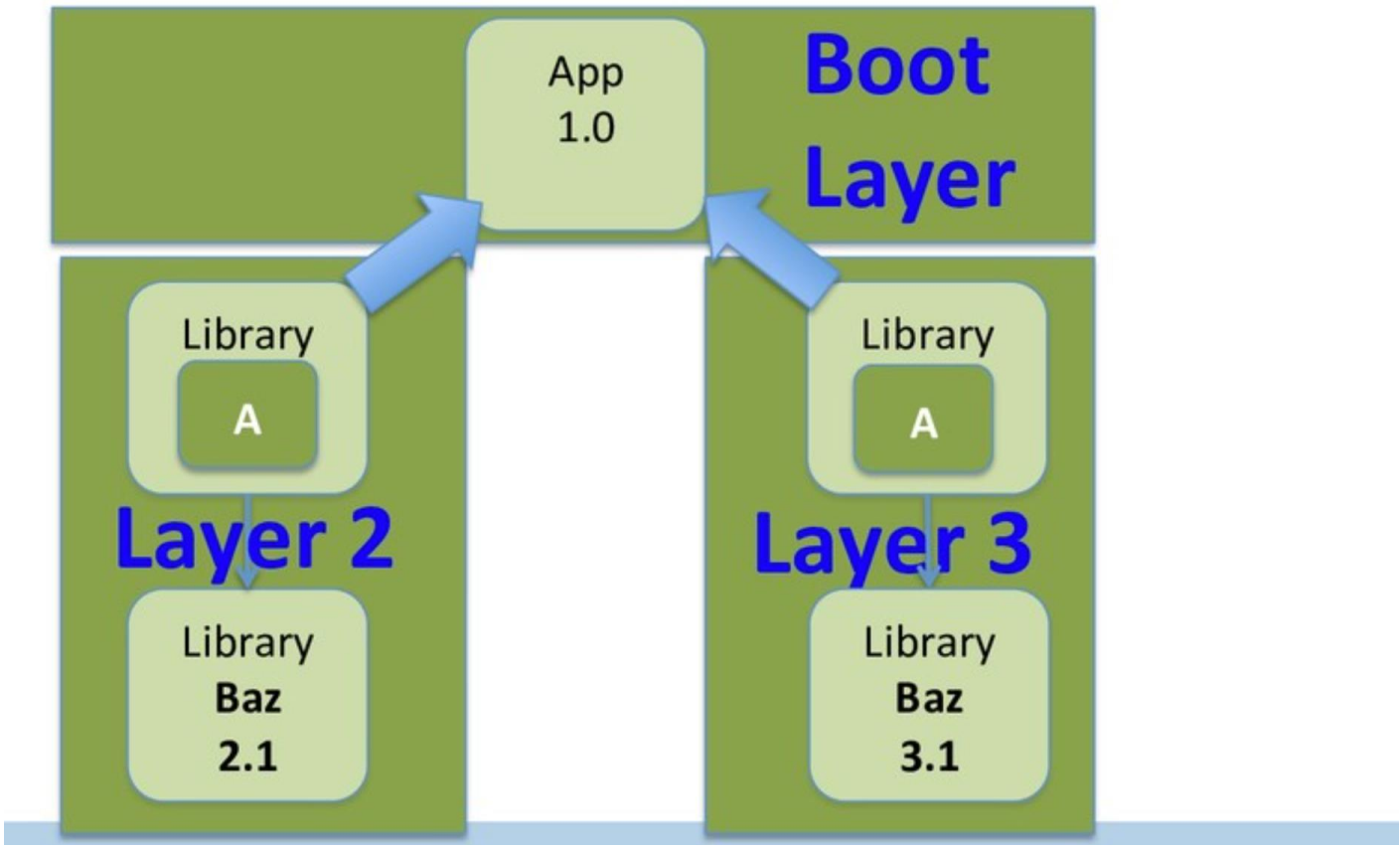
Jigsaw Layers Jar Hell Resolution



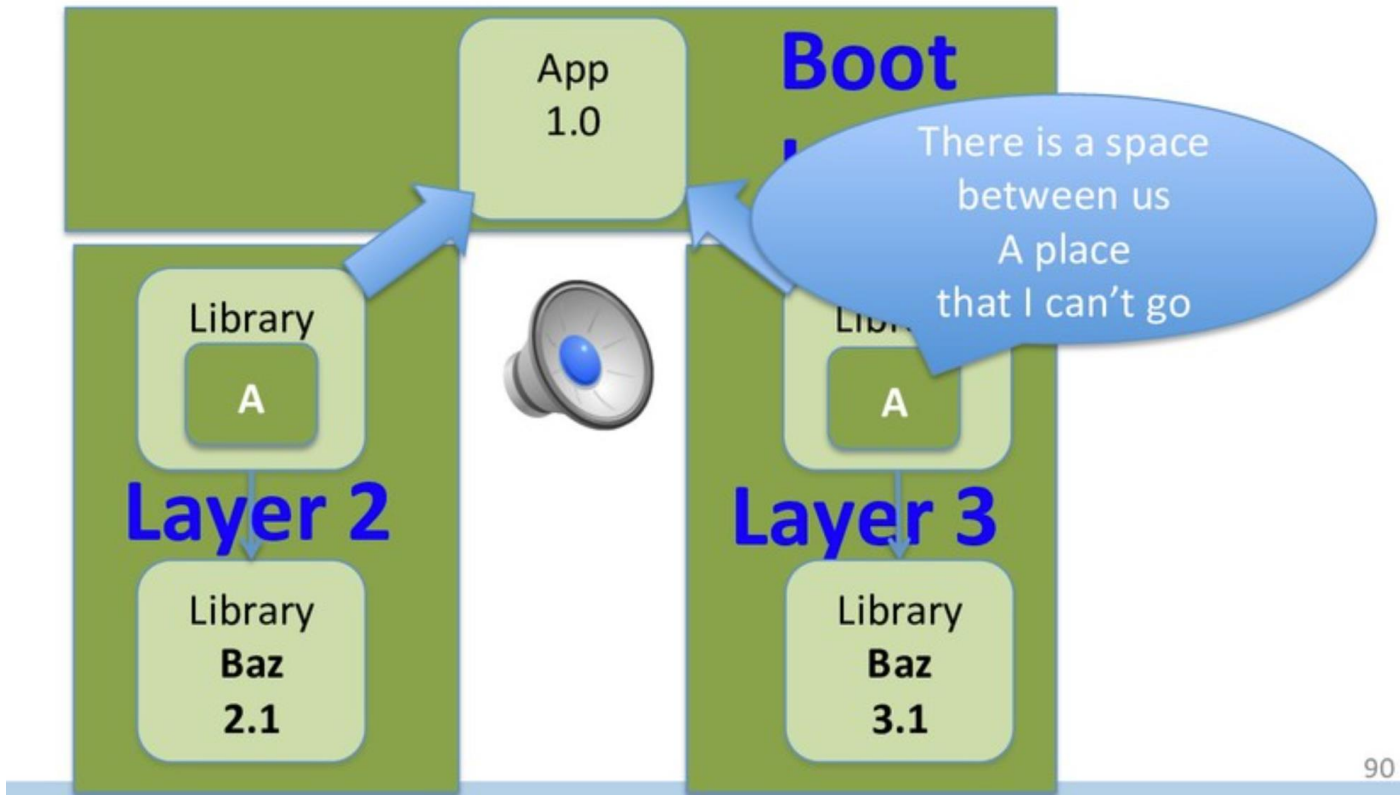
Jigsaw Layers Jar Hell Resolution



Jigsaw Layers Jar Hell Resolution



Jigsaw Layers Jar Hell Resolution



Jigsaw Layers

Question: if application modules cannot import child layers modules then how they can employ functionality implemented in these child layers?


IoC

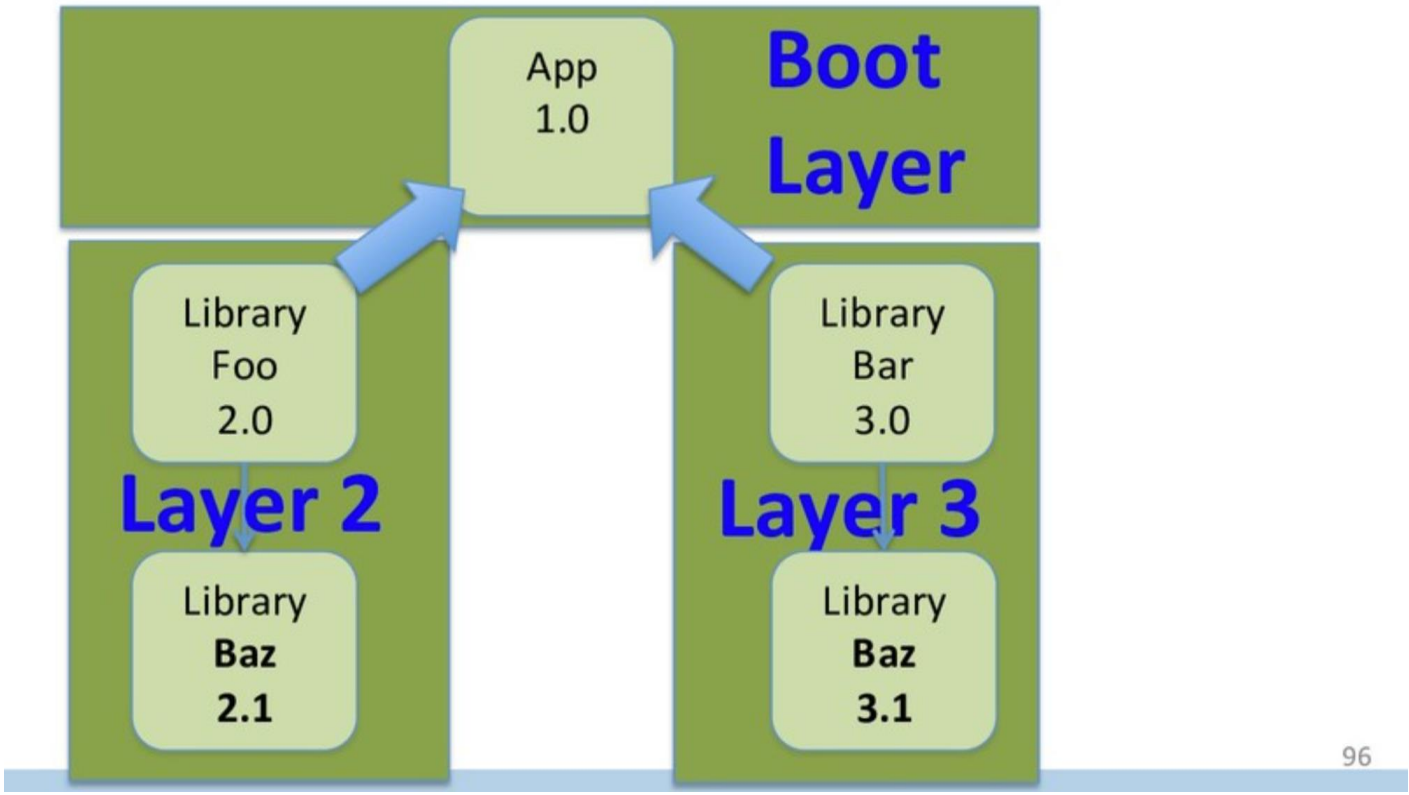
Jigsaw Layers

Question: if application modules cannot import child layers modules then how they can employ functionality implemented in these child layers?

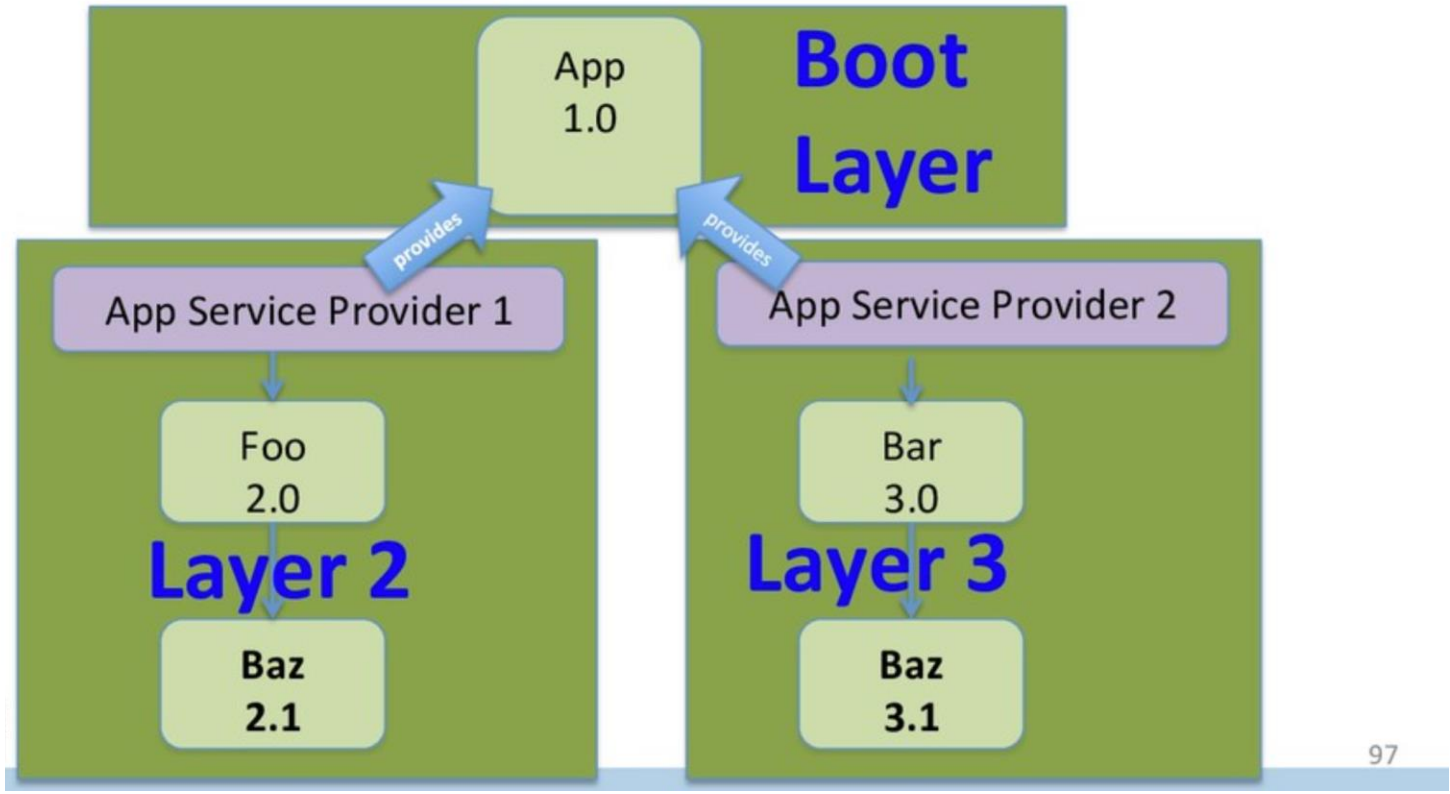
Answer: employ IoC design pattern via

Jigsaw Services: child layers can provide services to modules of parent layers

Jigsaw Layers Jar Hell Resolution



Jigsaw Layers Jar Hell Resolution




DEMO

How to use

```
var finder = ModuleFinder.of(Paths.get(“MyLayer”));
var parent = ModuleLayer.boot();
Configuration cf = parent.configuration().resolve(
    finder, ModuleFinder.of(), Set.of(“myRootMod"));
ModuleLayer myL = parent.defineModulesWithOneLoader(
    cf, ClassLoader.getSystemClassLoader());
var services = ServiceLoader.load(myL, MySrv.class);
```

Where else we can use layers

Problem: the servlet container standard knows nothing about modules so far

- Dependencies in war files are old plain classpath in fact!

Conclusion

Versions for modules look like a good idea but there are many problems with them:

- backward compatibility
- unsafety (classloaders constraints)
- NP-completeness on dependencies resolution

Conclusion

Jigsaw Layers allow to solve the Jar Hell problem safely:

- no classloaders constraints violations by construction
- Jigsaw services help employing functionality implemented in a child layer from a parent layer

Conclusion

Knowledge about the layers can help in modularizing of Java applications including server side applications